

Assignment 3: Multi-output learning

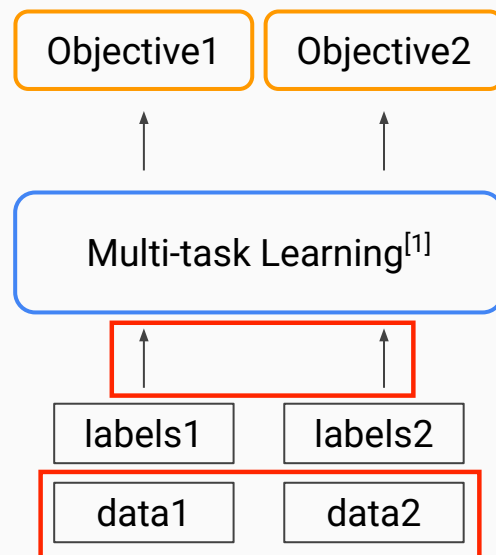
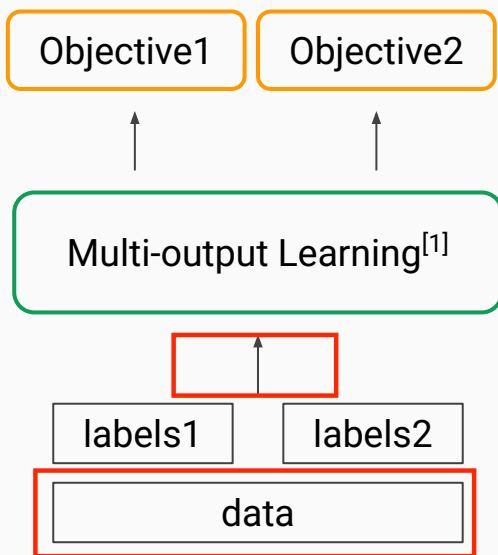
2025 NTHU Natural Language Processing

Hung-Yu Kao

IKM Lab TAs



Multi-output Learning



[1] Xu, Donna, et al. "Survey on multi-output learning." IEEE transactions on neural networks and learning systems 31.7 (2019): 2409-2429.

Assignment Description

- In Assignment 3, you will practice building a model for multi-output learning.
- You are going to use the **SemEval 2014 Task 1** dataset, where each data point is associated with multiple labels.

Dataset examples (SemEval 2014)

*Answer in red

Value: 1-5

sub-task1
regression

'0': NEUTRAL
'1': ENTAILMENT
'2': CONTRADICTION

sub-task2
3-class classification

premise	hypothesis	relatedness_score	entailment_judgement
A group of kids is playing in a yard and an old man is standing in the background	A group of boys in a yard is playing and a man is standing in the background	4.5	0
A man, a woman and two girls are walking on the beach	A group of people is on a beach	4.300000190734863	1

Dataset

*SemEval 2014 Task1 dataset

- Train split: 4500 pieces
- Validation split: 500 pieces
- Test split: 4927 pieces
- Each data piece: A premise, a hypothesis, a relatedness_score, an entailment_judgement

Code

Import packages

```
1 from transformers import BertTokenizer, BertModel
2 from datasets import load_dataset
3 from evaluate import load
4 import torch
5 from torch.utils.data import Dataset, DataLoader
6 from torch.optim import AdamW
7 from tqdm import tqdm
8 device = "cuda" if torch.cuda.is_available() else "cpu"
9 # You can install and import any other libraries if needed
```

You can install and import any other libraries if needed

Load the dataset from Hugging Face

You may cache the dataset at any location you prefer.

```
1 class SemevalDataset(Dataset):
2     def __init__(self, split="train") -> None:
3         super().__init__()
4         assert split in ["train", "validation", "test"]
5         self.data = load_dataset(
6             "sem_eval_2014_task_1", split=split, trust_remote_code=True, cache_dir="./cache/"
7         ).to_list()
8
9     def __getitem__(self, index):
10        d = self.data[index]
11        # Replace Chinese punctuations with English ones
12        for k in ["premise", "hypothesis"]:
13            for tok in token_replacement:
14                d[k] = d[k].replace(tok[0], tok[1])
15        return d
16
17    def __len__(self):
18        return len(self.data)
19
20 data_sample = SemevalDataset(split="train").data[:3]
21 print(f"Dataset example: \n{data_sample[0]} \n{data_sample[1]} \n{data_sample[2]}")
```

You need **datasets==2.21.0** to
download **sem_eval_2014_task_1**
from Hugging Face!

← Observe the data instances

TODO1: Create batched data with PyTorch DataLoader

```
1  # TODO1: Create batched data for DataLoader
2  # `collate_fn` is a function that defines how the data batch should be packed.
3  # This function will be called in the DataLoader to pack the data batch.
4
5  def collate_fn(batch):
6      # TODO1-1: Implement the collate_fn function
7      # Write your code here
8      # The input parameter is a data batch (tuple), and this function packs it into tensors.
9      # Use tokenizer to pack tokenize and pack the data and its corresponding labels.
10     # Return the data batch and labels for each sub-task.
11
12 # TODO1-2: Define your DataLoader
13 dl_train = # Write your code here
14 dl_validation = # Write your code here
15 dl_test = # Write your code here
```

`print(next(iter(dl_train)))`



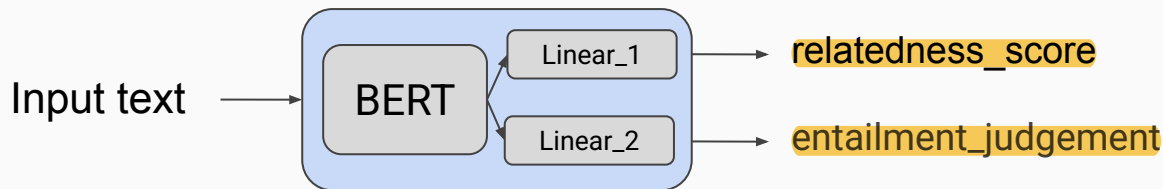
1. input_text (input_ids, token_type_ids, attention_mask)
2. labels1 (regression)
3. labels2 (classification)

TODO2: Construct your model

```
1 # TODO2: Construct your model
2 class MultiLabelModel(torch.nn.Module):
3     def __init__(self, *args, **kwargs):
4         super().__init__(*args, **kwargs)
5         # Write your code here
6         # Define what modules you will use in the model
7         # Please use "google-bert/bert-base-uncased" model (https://huggingface.co/google-bert/bert-base-uncased)
8         # Besides the base model, you may design additional architectures by incorporating linear layers, activation functions,
9         # Remark: The use of any additional pretrained language models is not permitted.
10    def forward(self, **kwargs):
11        # Write your code here
12        # Forward pass
```

You are required to build a model (using huggingface API) to handle the multi-output task. Please use bert-base-uncased model ([LINK](https://huggingface.co/google-bert/bert-base-uncased)).

Your model



An example of model achitecture (BERT model)

TODO3: Define optimizer, loss function, and dataloader

```
1  # TODO3: Define your optimizer and loss function
2
3  model = MultiLabelModel().to(device)
4  # TODO3-1: Define your Optimizer
5  optimizer = # Write your code here
6
7  # TODO3-2: Define your loss functions (you should have two)
8  # Write your code here
9
10 # scoring functions
11 psr = load("pearsonr")
12 acc = load("accuracy")
```

- We recommend using Adam or AdamW as the optimizer.
- For the loss functions, observe the type of each sub-task; use different loss functions for different types of tasks.

TODO4: Write the training loop

- Train your model using PyTorch gradient descend.
 - This time, you cannot use `HuggingFace Trainer`
- Compute gradient in each training step, and optimize the model
- The loss value is an aggregation of the losses from all sub-tasks.

```
1  best_score = 0.0
2  for ep in range(epochs):
3      pbar = tqdm(dl_train)
4      pbar.set_description(f"Training epoch [{ep+1}/{epochs}]")
5      model.train()
6      # TODO4: Write the training loop
7      # Write your code here
8      # train your model
9      # clear gradient
10     # forward pass
11     # compute loss
12     # back-propagation
13     # model optimization
```

TODO5: Evaluate your model

```
15     pbar = tqdm(dl_validation)
16     pbar.set_description(f"Validation epoch [{ep+1}/{epochs}]")
17     model.eval()
18     # TODO5: Write the evaluation loop
19     # Write your code here
20     # Evaluate your model
21     # Output all the evaluation scores (PearsonCorr, Accuracy)
22     pearson_corr = # Write your code here
23     accuracy = # Write your code here
24     # print(f"F1 Score: {f1.compute()}")
25
26     if pearson_corr + accuracy > best:
27         best = pearson_corr + accuracy
28         torch.save(model.state_dict(), f'./saved_models/best_model.ckpt')
```

- Evaluate the model on Validation set.
- For each instance, there are multiple labels for each sub-tasks. The evaluation result includes scores of all sub-tasks.
 - For relatedness_score, compute the Pearson correlation coefficients. For entailment_judgement, compute accuracy.
 - The sample code use torchmetrics package to compute scores.

TODO6: Test your model

```
1  # Load the model
2  model = MultiLabelModel().to(device)
3  model.load_state_dict(torch.load(f"./saved_models/best_model.ckpt", weights_only=True))
4
5  # Test Loop
6  pbar = tqdm(dl_test, desc="Test")
7  model.eval()
8
9  # TODO6: Write the test loop
10 # Write your code here
11 # We have loaded the best model with the highest evaluation score for you
12 # Please implement the test loop to evaluate the model on the test dataset
13 # We will have 10% of the total score for the test accuracy and pearson correlation
```

Load the model checkpoint that achieves the best evaluation score and run it on test set.
Then compute the test Pearson correlation coefficients and accuracy

Coding work : 40%

TODOs	Scores
TODO1: Create batched data with PyTorch DataLoader	5%
TODO2: Construct your model	5% (Your submitted code must use the bert-base-uncased model; otherwise, you will not receive this 5%)
TODO3: Define optimizer, loss function, and dataloader	5%
TODO4: Write the training loop	5%
TODO5: Evaluate your model	10%
TODO6: Test your model	10%

Submission

Scoring

Coding work : 40%

Score: 10% (The higher the accuracy achieved, the higher the score awarded.)

- A screenshot must be included at the end of the report.
- Baseline: (Pearson 0.8, Accuracy 0.8, **test set**, 4%)

Report: 50%

- Train a RoBERTa-base model on the same dataset and report the performance of both BERT-base and RoBERTa-base models. (10%) Discuss the main differences between BERT and RoBERTa, and explain how these differences may affect performance. (5%)
- **Train a GPT-2** model on the same dataset and evaluate its performance. (10%) What are the main difference of BERT and GPT2? Discuss the main differences between BERT and GPT-2, and analyze how these differences influence their performance. (5%)
- Compare the performance of a multi-output learning model with BERT models trained separately on each sub-task. Does multi-output learning improve performance? Explain why or why not. (5%)
- Why does your model fail to correctly predict some data points? Please provide an error analysis and introduce how do you improve your performance. (10%)
- ... **Anything that can strengthen your report.** (5%)

Delivery policies: File formats

- Coding work: Python file (.py)
 - Download your script via Colab.
- Package list: requirements.txt
 - E.g., `numpy==1.26.3`
- Report: Microsoft Word (.docx)
- **No other formats are allowed.**
- Zip the files above before uploading your assignment.



Delivery policies: Filenames

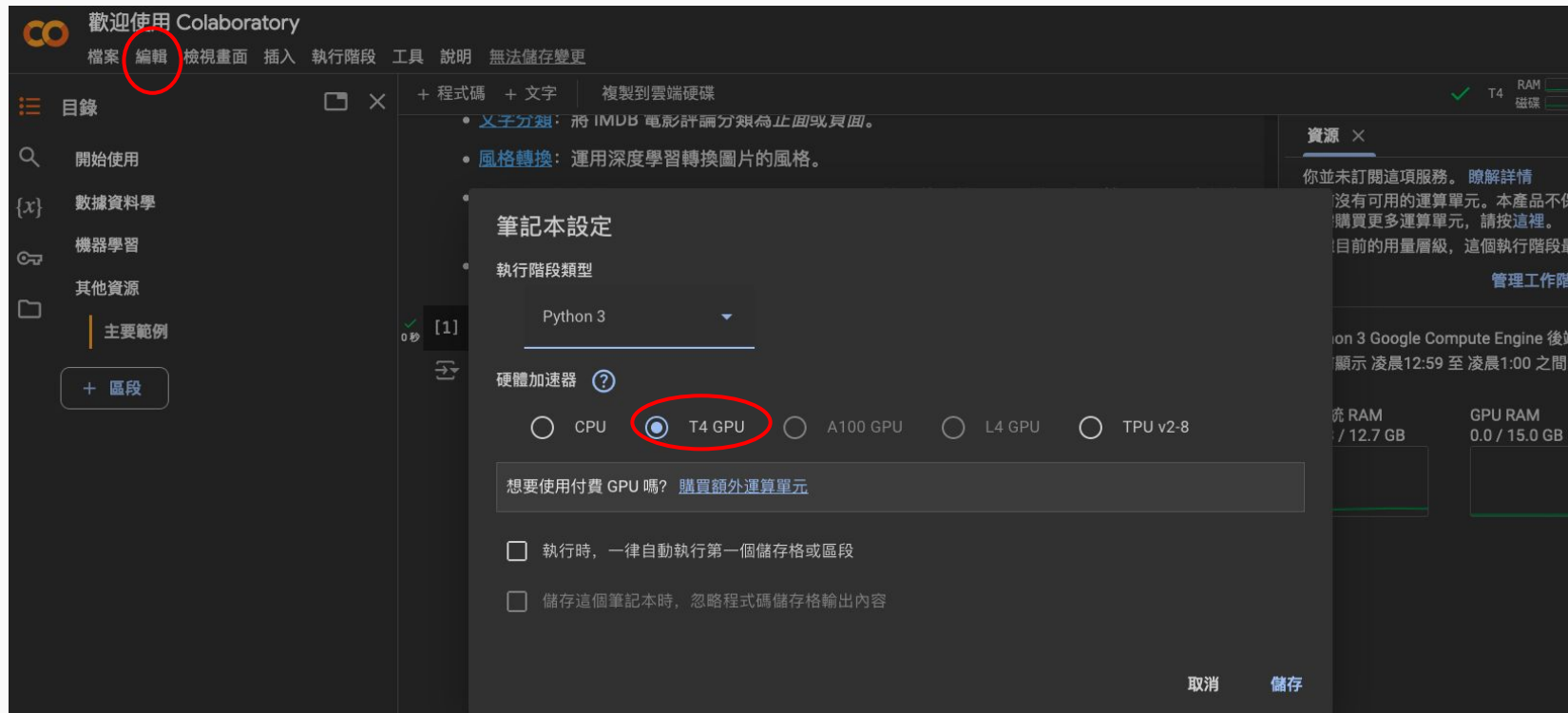
	Filename rule	Filename example
Coding work	NLP_HW3_ school _student_ID.py	NLP_HW3_ NTHU _12345678.py
Report	NLP_HW3_ school _student_ID.docx	NLP_HW3_ NTHU _12345678.docx
Package list	requirements.txt	
Zipped file	NLP_HW3_ school _student_ID.zip	NLP_HW3_ NTHU _12345678.zip

Delivery policies: Things You should include

- In your report:

	Example	
Environment types	If Colab or Kaggle	If local
Running environment	Colab	System: Ubuntu 22.04, CPU: Ryzen 7-7800X3D
Python version	Colab	Python 3.10.1
GPU(s) you used	Please check the info from Colab	NVIDIA RTX 3090 * 1

How to check the allocated GPU on Colab?



Type in a code block: `!nvidia-smi`

Delivery policies: Rules of coding

- If you use ChatGPT or Generative AI, please specify your usage **both** in:
 - **Code comments**
 - **Reports**
- **No plagiarism.** You should not copy and paste from your classmates. **Submit duplicate code or report will get 0 point !**
- Please provide links if you take the code from the Internet as reference.
- The following behaviors **will cause loss in the score of the assignment:** (1) **Usage with Generative AI without specifications** (2) **Internet sources without specifications** (3) **Plagiarism.**

Punishments

Rule	Name your code: NLP_HW3_ school _st udent_ID.py (only .py is acceptable)	Name your report: NLP_HW3_ school _stu dent_ID.docx	Name your file: NLP_HW3_ school _stu dent_ID.zip	Include requirements.txt
Punishment	-5	-5	-5	-5
Rule	Include python version in your report	Do not modify the code template (only changes to data loading are allowed).	Do not modify the report template	Your code or report should not shows a high degree of similarity to another student's submission.
Punishment	-5	-5	-5	-100 for both

If you are using Colab, go to **File** → **Download** → **Download .py** to obtain the Python file.

Uploading the zipped file

- Please upload your file to NTU COOL.
- You will have three weeks to finish this assignment.
- If you have any question, please e-mail to **nthuikmlab@gmail.com**